LINUX ASYNC I/O / ZIG 0.17.0

Zigで覗く、Linuxの非同期I/O

poll・epoll・io_uring を、動かして見る

いつも使う async/await の、少し下へ。

async/awaitの下では何が起きている？

レスポンスを待つ間、
「データが届いた」と誰が教えてくれる？



典型的なネットワークI/Oの一例。言語・runtime・操作によって経路は異なる。

その下にあるLinuxの仕組みを、直接見てみる

Zigとは？

Cに近い低レイヤーを扱える
システムプログラミング言語

メモリやリソースを明示的に管理する

今回は std.os.linux

libcを経由せず、Linuxの
システムコールを呼び出す

ミニデモ：write

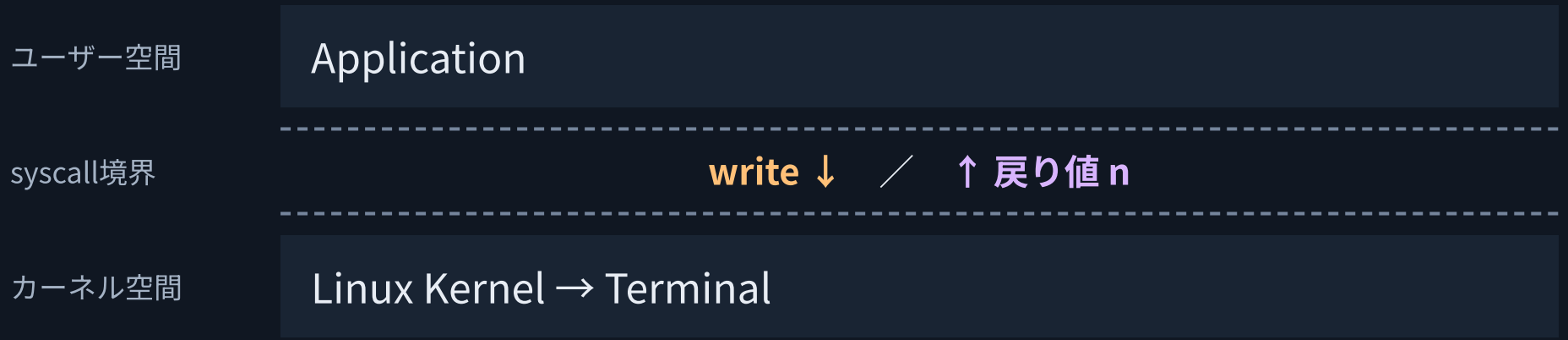
write → hello → 戻り値

ターミナルへ ↗

システムコールは、OSへの依頼

```
const n = linux.write(1, "hello\n", 6);
```

1：stdoutのfd　hello：データ　6 byteを依頼 → n byteを書き込んだ



Zigからlibcを経由せず、Linuxへ処理を依頼する

fdは、プロセスが持つ番号

0 / 1 / 2

stdin / stdout / stderr

そのほかの番号

ファイル / pipe / socket …

プロセスごとのfdテーブルで管理される。

pipe：データの通り道

書き込み側の**fd**

↓ **hello**

読み取り側の**fd**

ミニデモ：blocking read

データなし　→ readが待つ →　**到着後にhello**

ターミナルへ ↗

データがないとき、どう待てばいい？

blocking read

このfdに届くまで
スレッドが待つ

non-blocking read

まだ届いていなければ
EAGAINで戻る

fd A ?　　fd B ?　　fd C ?　　…　　何度も調べ続ける？

どのfdが読み取り可能になったか、まとめて知りたい

poll

pollは、fdがReadyになるのを待つ

```
const count = linux.poll(&fds, fds.len, 3000);
```

fds：監視fdの一覧　**revents：各fdに起きたイベント**

Application	依頼 → / 結果 ←	Linux Kernel
fd一覧 A・Bを毎回渡す	→	**poll**：Ready状態を確認
reventsを確認する	←	**fd B がReady**
read(fd B) → hello	⇄	データはここで読み取る

pollはデータを読まない。Readyを受け取ってからread

2本のpipe、届くのは片方だけ

01 空のpipeでは **EAGAIN**

02 **poll** が返すのは **Readyなfd**

03 そのあと **read** すると **hello**

ターミナルへ ↗

Readyが返っても、まだreadしていない

epoll

epollは、初期登録と待機を分ける

```
const created = linux.epoll_create1(linux.EPOLL.CLOEXEC);
_ = linux.epoll_ctl(epfd, linux.EPOLL.CTL_ADD, fd, &event);
const count = linux.epoll_wait(epfd, &events, events.len, 3000);
```

呼び出しの抜粋。fdの変換・戻り値の確認はデモ全文で。

Application	依頼 → / 結果 ←	Linux Kernel
create1 → ctl ADD	→	**監視集合 A・B を保持**
epoll_waitで待つ	⇄	**fd B がReady**
read(fd B) → hello	⇄	データはここで読み取る

最初に登録し、以後は監視fd一覧を渡し直さず待つ

同じpipe、同じhelloで見てみる

01 **epoll_ctl ADD** で **2つのfd**を登録

02 **epoll_wait** が **Readyなfd**を返す

03 **read** で **hello**を取得

ターミナルへ ↗

登録は最初の一度。waitでReadyを受け取る

io_uring

io_uringは、操作を頼んで完了を受け取る

```
_ = try ring.read(42, pipe[0], .{ .buffer = &buffer }, 0);
_ = try ring.submit();
const cqe = try ring.copy_cqe();
```

std.os.linux.IoUring：低レベルラッパー。エラー処理はデモ全文で。

Application	共有リング	Linux Kernel
READのSQEを準備	→	SQ：user_data=42
submit → io_uring_enterで通知	→	**pipeのfd**から読み取る
CQE取得 → **bufferにhello**	←	CQ：user_data=42 / res=5

res は読み取ったbyte数、または負のerrno

READを投入すると、何が返る？

01 **SQE** に **pipeのfd**と **42**を設定

02 **submit** して、データの到着を待つ

03 **CQE** は **42 / 5 byte**、bufferには **hello**

ターミナルへ ↗

READを投入。CQEが来たらbufferにhello

「読み取れる」と「読み取った」の違い

poll / epoll：Readiness

読める状態のfdが分かる

↓

アプリが**read**する

↓

helloを取得

io_uring READ：Completion

READ操作を投入する

↓

カーネルが読み取りを処理する

↓

完了結果と**hello**を取得

通知されるのは、状態か、操作の完了か

同じhelloを受け取る、3つの方法

poll	epoll	io_uring READ
1 fd一覧を渡す	1 fdを事前登録	1 READを依頼
↓	↓	↓
2 Ready通知	2 Ready通知	2 CQE：操作完了
↓	↓	↓
3 readする	3 readする	3 bufferを確認
hello	hello	hello

一覧か登録か。Readyの通知か、操作の完了か。

短いプログラムでも、役割が見える

01 **fd**は、カーネルへの依頼で使う番号

02 **poll / epoll**は、複数のI/Oを待つ道具

03 **io_uring**は、操作と完了をキューでやり取りする

straceで見えるのはsyscallの境界。共有メモリ内の操作までは見えない。

非同期処理の、少し下を覗いてみた

01 async/await の下には、OSのI/O機構がある

02 Ready通知とCompletion通知では、依頼の仕方が違う

03 Zigとstraceで、その動きを直接確かめられる

普段何気なく使う非同期処理。
その少し下に、Linuxの面白い仕組みがある。

デモと参考資料

短いZigプログラム

- github.com/moribit/zig-syscall-lt / demos
- Zig 0.17.0 Documentation

非同期処理の実装例

- libuv：Design overview
- Python：Event Loop

過去のベンチマーク・分析・旧スライドはリポジトリ内に保存。

Linux man-pages

- syscalls(2) / pipe(7)
- poll(2)
- epoll(7)
- io_uring(7)